

Homework 1

Haozheng Wang

September 2025

1 Short questions

1.1 Homogeneous coordination

1.1.1

Solution:

$$\frac{x_i}{w} = \frac{x'_i}{1}$$
$$x'_i = \frac{x_i}{w}$$

Notice that $w = 0$ makes the denominator meaningless. By drawing an analogy between this situation and the derivation of vanishing point, we assert that $w = 0$ represents a point at infinity.

1.1.2

Answer:

1. Using homogeneous coordinates provides us with a more natural way to deal with translation, i.e. using matrix multiplications uniformly instead of making additions. This change between coordinate systems is an essential prerequisite for combining rotation and translation together as the extrinsic camera parameters matrix.

Before:

$$R_{3 \times 3} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} + T_{3 \times 1} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

After:

$$\begin{bmatrix} R_{3 \times 3} & T_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

2. Homogeneous coordination enables us to handle the transformation between camera coordinates and image coordinates more conveniently.

To be more explicit:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{z} \begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where factor $\frac{1}{z}$ is considered the cause for the projection's non-linearity and for the inconvenience in calculation to some extent (since camera coordinate z is an intermediate variable). However, by using the properties of homogeneous coordinates, this awful factor can be formally omitted:

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3. As we have seen in the derivation of vanishing point, homogeneous coordinates enable us to represent points at infinity.

In computational photography, homogeneous coordinates are used because they (1) unify geometric transformations into a single projection matrix and (2) extend space to include points at infinity. These make homogeneous coordinates essential in computational photography.

1.2 Dolly zoom

Answer:

Technically, a "dolly zoom" is produced by moving the position of the camera and modifying its focal length at the same time. There are two ways to realize this effect:

(1) When the camera approaches the object, the photographer decreases the focal length of the camera, thus making FoV larger (which is actually the case in Figure 1).

(2) When the camera pulls away from the object, the photographer increases the focal length of the camera, thus making FoV smaller.

Using these methods, it is possible that the size of a specific object in the image remains unchanged, while objects in the background undergo dramatic changes.

2 Camera parameters from an image

2.1

Solution:



Figure 1: A figure of our campus with auxiliary lines displayed

Note 1: We draw two pairs of horizontal parallel lines in the picture to determine the vanishing line. One pair runs along the direction of the edge of the ground tiles, while the other pair runs along the direction of the diagonal of four ground tiles (which is not so obvious from direct observation of the image).

Note 2: It is noticed that a number of solutions previously pretended to have found the vanishing line by using "two pairs" of horizontal parallel lines, but in fact, they only used one bunch of four horizontal parallel lines, which is not an appropriate approach to this question.

According to our measurement, the y-coordinate of the vanishing line is 1745 pixels, the y-coordinate of the top of the flower bed (Point A) is 1949 pixels, and the y-coordinate of the bottom of the flower bed is 2143 pixels. Thus

$$H = \frac{2143 - 1745}{2143 - 1949}h = 1.56m$$

2.2

Solution:

First we write down the intrinsic parameter matrix:

$$\begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

in which $c_x = \frac{4096}{2} \text{pixels} = 2048 \text{pixels}$, $c_y = \frac{3072}{2} \text{pixels} = 1536 \text{pixels}$.

We then move on to construct the extrinsic parameter matrix. The rotation matrix needs to be written as two matrices multiplied together. The first part can be written according to the right-handed world coordinate system shown in the picture and the image coordinate system, whose x direction runs to the right, y direction runs downward and z direction runs into the image vertically; That is, $x = Y, y = -Z, z = -X$. The second part represents a rotation around x-axis, whose rotation angle needs to be expressed in terms of focal length f :

$$R = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & -\frac{\delta}{f} & 0 \\ 0 & \frac{\delta}{f} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $\delta = 1745 - 1536 = 209$ pixels.

The translation matrix moves the position of the camera to the original point:

$$T = \begin{bmatrix} 1 & 0 & 0 & -13.4 \\ 0 & 1 & 0 & -4.5 \\ 0 & 0 & 1 & -1.56 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Now we can combine these results together to figure out the projection, and then apply it to Point A to obtain the equations we need:

$$w \begin{bmatrix} 1892 \\ 1949 \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 2048 & 0 \\ 0 & f & 1536 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ \frac{209}{f} & 0 & -1 & 0 \\ -1 & 0 & -\frac{209}{f} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -13.4 \\ 0 & 1 & 0 & -4.5 \\ 0 & 0 & 1 & -1.56 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 3.74 \\ 0.76 \\ 1 \end{bmatrix}$$

We extract the first and third dimensions to form the system of equations:

$$-0.76f + 27443.2 + \frac{342425.6}{f} = 1892w \quad (1)$$

$$13.4 + \frac{167.2}{f} = w \quad (2)$$

Solving the equations, we obtain $f = 2763$ pixels.

3 Image filtering and subsampling

3.1 Image Filtering

Denote $I(i, j)$ as a $w \times h$ digital image and $h(u, v)$ as a $m \times n$ kernel. For simplification, introduce $p = \lfloor \frac{m}{2} \rfloor, q = \lfloor \frac{n}{2} \rfloor$. Then we can derive the formulation of correlation:

$$I(i, j) \leftarrow \sum_{k=-p}^p \sum_{l=-q}^q I(i+k, j+l)h(k, l)$$

The function *cross_correlation_2d* can be realized in a straightforward way:

```
1 def cross_correlation_2d(input_image_array, filter):
2     w = input_image_array.shape[0]
3     h = input_image_array.shape[1]
4     m = filter.shape[0]
5     n = filter.shape[1]
6     p = m // 2
7     q = n // 2
8     output_image_array = np.zeros_like(input_image_array, dtype='
9         float64')
10    for i in range(w):
11        for j in range(h):
12            for k in range(-p, p + 1):
13                for l in range(-q, q + 1):
14                    if 0 <= i + k < w and 0 <= j + l < h:
15                        output_image_array[i][j] +=
                            input_image_array[i + k][j + l] *
                            filter[k][l]
16    return output_image_array
```

The calculation of convolution is just a simple step from correlation:

$$I(i, j) \leftarrow \sum_{k=-p}^p \sum_{l=-q}^q I(i-k, j-l)h(k, l)$$

Thus, we only need to transpose the input filter and directly call *cross_correlation_2d*.

```
1 def convolve_2d(input_image_array, filter):
2     return cross_correlation_2d(input_image_array, np.transpose(
3         filter))
```

As a sanity check, we test these two functions in a box-filter case:

```
1 # cross_correlation_2d
2 input_image = Image.open('Lena.png')
3 input_image_array = np.array(input_image)
4 box_filter = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]]) / 9
5 output_image_array = cross_correlation_2d(input_image_array,
6     box_filter)
```

```

6 |     output_image = Image.fromarray(np.uint8(output_image_array))
7 |     output_image.save('Lena_Correlation.png')
8 |
9 |     # convolution_2d
10 |    input_image = Image.open('Lena.png')
11 |    input_image_array = np.array(input_image)
12 |    box_filter = np.array([[1, 1, 1], [1, 1, 1], [1, 1, 1]]) / 9
13 |    output_image_array = cross_correlation_2d(input_image_array,
14 |                                              box_filter)
15 |    output_image = Image.fromarray(np.uint8(output_image_array))
    output_image.save('Lena_Convolution.png')

```

And the results are as follows:



Figure 2: Image Filtering

3.2 Gaussian Blur

$$d(i, j; k, l) = \exp\left(-\frac{(i - k)^2 + (j - l)^2}{2\sigma_d^2}\right)$$

For Gaussian blur kernel, each term in the array is proportional to the exponential term above, where the distance is calculated from the central point of the kernel to the targeted point. Remember a normalization is required before we return the kernel.

```

1 | def gaussian_blur_kernel_2d(kernel_size, standard_deviation):
2 |     p = kernel_size // 2
3 |     gaussian_kernel = np.zeros((kernel_size, kernel_size))
4 |     for i in range(kernel_size):
5 |         for j in range(kernel_size):
6 |             gaussian_kernel[i][j] = np.exp(-((i - p) * (i - p) + (j
7 |             - p) * (j - p)) / (2 * standard_deviation *
8 |             standard_deviation))
    gaussian_kernel /= np.sum(gaussian_kernel)
    return gaussian_kernel

```

3.3 Low Pass Filters

For this part, we first generate a Gaussian blur kernel by calling *gaussian_blur_kernel_2d* with *kernel_size* and *standard_deviation*. After obtaining the filter, we pass *input_image_array* and filter to *convolve_2d*, thus filtering out details with high frequencies from the image.

```
1 def low_pass(input_image_array, kernel_size, standard_deviation):  
2     filter = gaussian_blur_kernel_2d(kernel_size,  
3         standard_deviation)  
4     out_image_array = convolve_2d(input_image_array, filter)  
5     return out_image_array
```

The results are as follows:



(a) Lena

(b) Lena_Low_Pass

Figure 3: Low Pass

3.4 Image subsampling/downsampling

The function *image_subsampling* can be realized by simply keeping pixels whose horizontal and vertical coordinates are both even numbers and throwing away pixels whose coordinates are not.

```

1 def image_subsampling(input_image_array):
2     w = input_image_array.shape[0]
3     h = input_image_array.shape[1]
4     c = input_image_array.shape[2]
5     w_tilda = w // 2
6     h_tilda = h // 2
7     output_image_array = np.zeros((w_tilda, h_tilda, c))
8     for i in range(w_tilda):
9         for j in range(h_tilda):
10             output_image_array[i][j] = input_image_array[2 * i][2 *
11                 j]
12     return output_image_array

```

The results are as follows:



Figure 4: Image subsampling/downsampling

3.5 Gaussian pyramid

For each level of Gaussian pyramid, we firstly process the input image with *low_pass* to smooth out those fine details with high frequencies. Then we subsample the blurred image to acquire a smaller image with half resolution of the original image. Notice that we set *kernel_size*=3 and *standard_deviation*=1 by default, which will be passed into *low_pass* to generate the Gaussian blur kernel.

```

1 def gaussian_pyramid(input_image_array, pyramid_level):
2     pyramid = [input_image_array]
3     for i in range(pyramid_level - 1):
4         blur_image_array = low_pass(input_image_array, 3, 1)
5         subsample_image_array = image_subsampling(blur_image_array)
6         pyramid.append(subsample_image_array)
7         input_image_array = subsample_image_array

```

```
8 |         return pyramid
```

An auxiliary function *output_gaussian_pyramid* is constructed to make our life easier, which uses some tricks to finish all the tasks automatically:

```
1 | def output_gaussian_pyramid(file_name , pyramid_level):
2 |     input_image = Image.open(f'{{file_name}}')
3 |     input_image_array = np.array(input_image)
4 |     pyramid = gaussian_pyramid(input_image_array , pyramid_level)
5 |     file_name , extension = file_name.split('.')
6 |     sequence = [None, 'half', 'quarter', 'one-eighth']
7 |     for i in range(1, pyramid_level):
8 |         output_image_array = pyramid[i]
9 |         output_image = Image.fromarray(np.uint8(output_image_array)
10 |         )
11 |         output_image.save(f'{{file_name}}_{{sequence[i]}}.{{extension}}')
```

The results are shown in the following figures:



Figure 5: Gaussian Pyramid of Lena



Figure 6: Gaussian Pyramid of the frog



Figure 7: Gaussian Pyramid of Yanami Anna